

AI辅助技术穿刺代码实战总结

AI辅助技术穿刺代码实战总结

一、项目背景与挑战

项目背景

公司当前配置器在反向推理及最优配置推荐方面存在能力不足。前期通过初步技术洞察，发现约束求解可能是解决方案之一。公司决定采用实际业务场景进行技术打样，完成可运行的Demo开发，包括：

- 约束求解的业务建模
- 约束求解的实现代码生成
- 约束求解执行器开发

主要挑战

1. 技术门槛高

约束求解涉及约束理论、求解器等专业知识，技术上手难度大，需要深入理解相关理论基础。

2. 团队经验不足

- 团队成员具备1.5名Java编程基础，熟悉面向对象编程和建模思路
- 但对于约束求解技术，所有成员都是第一次接触，存在较大的不确定性

3. 时间紧迫

项目要求在1.5个月内完成从业务建模、模型实现、求解器实现的完整开发，并完成一个实际产品的打通，需要实现IT人员和数据开发人员的高效协同。

按照以往项目经验，对于这种基础理论不熟悉、又要实现系统原型的情况，通常需要3到6个月才能完成。

二、项目成果

1. 技术突破

首次在项目组引入约束求解技术，完成了一个实际产品从业务建模、模型开发、测试、发布到应用的全流程打通。

2. 代码产出

完成代码约2.5万行，实现了完整的约束求解系统。

3. 项目效率

项目高效完成，得益于在各个环节充分利用AI辅助开发。本文将重点分享在新技术项目中，如何与AI这个“硅基伙伴”高效协作，以及作为主观能动的人应该扮演什么角色。

三、AI辅助实践经验

初期阶段：建立知识基础，AI作为导师

核心策略：关联学习法

在项目初期，团队成员对约束求解技术完全陌生。此时AI的作用类似于导师，但需要注意：

- **如何高效学习**：采用关联学习法，通过提示词构建知识图谱，快速建立知识网络

约束编程中是如何表达Java高级语言中的场景语法（如：赋值语句），请场景，逐个说明，按下列要求输出

一、输出的使用java库位：ortools-java 9.12.4544

二、输出的格式为：

场景1: xxx（如：变量与约束的直接映射）

和java的相同点和不同点： xxx（如：都有变量，都有常量）

案例： xxx（如: $x = y + 5$ ）

MiniZinc约束表达:xxx（如:constraint $x = y + 5$;)）

Java高级语言表达： xxx， 如：

```
int x=1;
int y=2;
x = y+ 5;
```

三、java输出的代码分两部分

//依赖的包:

//每个例子包装为一个函数

```
public static void testCase1_xxx(){
    //xxxx
```

```
}
```

场景2: 条件赋值 (If-Then-Else)

和Java的相同点和不同点

- 相同: 都能表达条件逻辑
- 不同: CP中使用布尔变量和蕴含约束, 而非流程控制语句

案例

```
if (a > b) then x = 10 else x = 5
```

MiniZinc约束表达

javascript

 复制  下载

```
constraint (a > b) -> (x == 10) /\ (a <= b) -> (x == 5);
```

Java高级语言表达

java

 复制  下载

```
int a=7, b=3;  
int x = (a > b) ? 10 : 5; // 条件运算 (x=10)
```

ORTools约束表达

java

 复制  下载

```
public static void testCase2_ConditionalAssignment() {  
    CpModel model = new CpModel();  
    IntVar a = model.newIntVar(0, 10, "a");  
    IntVar b = model.newIntVar(0, 10, "b");  
    IntVar x = model.newIntVar(0, 10, "x");  
  
    // 创建条件布尔变量  
    BoolVar cond = model.newBoolVar("cond");  
    model.addGreaterThanOrEqual(a, b).onlyEnforceIf(cond);  
    model.addLessOrEqual(a, b).onlyEnforceIf(cond.not());  
  
    // 条件赋值  
    model.addEquality(x, 10).onlyEnforceIf(cond);  
    model.addEquality(x, 5).onlyEnforceIf(cond.not());  
  
    System.out.println("约束: if(a>b) x=10 else x=5");  
}
```

真的要懂了，后续才能使用

ORtool 的addImplication是什么意思?

🧠 已深度思考 (用时 11 秒) ▾

🔍 已阅读 8 个网页 🧑🏠🌐🔗

OR-Tools 中的 `AddImplication` 方法用于在约束编程 (CP-SAT) 模型中添加逻辑蕴含关系约束，即实现 "如果A成立，那么B必须成立" 这样的逻辑关系。它在定义条件约束时非常有用。

🧠 理解 AddImplication

`AddImplication` 用于在两个布尔变量之间建立逻辑关系。它的基本形式通常是：

python

```
model.AddImplication(a, b)
```

📄 复制 📄 下载

这表示：如果布尔变量 `a` 为 `True` (成立)，那么布尔变量 `b` 也必须为 `True` (成立)。

除了这个，两个布尔变量之间逻辑关系还有哪些，请介绍

📊 总结表

逻辑关系	含义	OR-Tools API (主要方法)
Implication	如果 a 那么 b	<code>model.AddImplication(a, b)</code>
AND	a 和 b 都真	<code>model.AddBoolAnd([a, b])</code>
OR	a 或 b 至少一个真	<code>model.AddBoolOr([a, b])</code>
XOR	a 和 b 恰好一个真	<code>model.AddBoolXOr([a, b])</code>
Equivalence	a 当且仅当 b	<code>model.Add(a == b)</code>
NAND	a 和 b 不同时为真	<code>model.AddBoolOr([a.Not(), b.Not()])</code>
NOR	a 和 b 都假	<code>model.AddBoolAnd([a.Not(), b.Not()])</code>

-
- **AI偶尔不太靠谱：**对AI提供的代码和方案，必须实际运行验证 (Run the code)，通过读代码，调代码深入理解其原理

生成的代码不可运行（使用不存在的api等，版本不匹配等）

```
src > main > java > J ConfigServerSimple.java > ⚡ ConfigServerSimple > 📦 main(String[])

8 public class ConfigServerSimple {
9     public static void main(String[] args) {
29
30         // 计算总内存容量和总内存条数
31         IntVar totalMemCapacity = model.newIntVar(lb: 0, ub: 512, name: "total_mem_capacity");
32         model.addEquality(totalMemCapacity,
33             LinearExpr.weightedSum(new IntVar[]{mem16Count, mem32Count}, new long[]{16, 32}));
34
35         IntVar totalMemBars = model.newIntVar(lb: 0, ub: 32, name: "total_mem_bars");
36         model.addEquality(totalMemBars,
37             LinearExpr.sum(new IntVar[]{mem16Count, mem32Count}));
38
39         // ===== 添加约束条件 =====
40
41         // 约束1: 使用场景约束 (办公场景要求CPU频率≥2.6GHz)
42         IntVar isWork = model.newBoolVar(name: "is_work");
43         model.addEquality(scenario, value: 0).onlyEnforceIf(isWork);
44         model.addNotEqual(scenario, 0).onlyEnforceIf(isWork.not());
45         model.addEquality(cpuChoice, value: 1).onlyEnforceIf(isWork); // 只有cpu_002满足2.6GHz
46     }
```

- **克服AI的不准确性：**
 - 对AI提供的代码和方案，必须实际运行验证（Run the code）
 - 快速建立知识框架，形成大概印象后，知道如何提出更专业的问题
 - 在领域内，能够提出类似问题的能力是关键

实践要点：

- 通过关联学习法构建知识图谱
- 对AI的输出保持怀疑态度，必须验证
- 快速建立领域知识框架，提升提问质量

中期阶段：同频共振，深度协作

核心策略：专业提问，专业回答

当团队对约束求解技术有了基本了解后，与AI的协作进入新阶段：

- **同频共振：**能够提出专业的问题，获得专业的回答，通过不断学习得到线索，了解到Tacton在求解器可实现复杂产品约束求解实现原理，根据自己从官网的[\[基于规则系统高效求多解策略 - DeepSeek\]\(https://chat.deepseek.com/a/chat/s/80e4a539-51e5-401d-a49e-03039084fe5e\)](https://chat.deepseek.com/a/chat/s/80e4a539-51e5-401d-a49e-03039084fe5e)

当前的ParaVar.var定义为IntVar，如果是String类型参数或多选型参数，应该名字建模呢，请参考一下SAP Variant、ConfigIT、Tacton等配置器的建模方式给出答案，结合案例来说明



已深度思考 (用时 18 秒)

我主要参考了行业常见的配置器实践（如SAP Variant Configuration ⁴、Tacton CPQ ³ 以及其他产品配置管理概念 ⁷），为你提供 String 及多选型参数的建模思路和基于 OR-Tools 的实现示例。

参数类型扩展建模方案

参数类型	存储变量类型	选项选择变量	特点
整数型 (Int)	IntVar	Map<Integer, BoolVar> (选项ID → 是否选中)	选项
字符串型 (String)	IntVar (编码值)	Map<Integer, BoolVar> (选项ID → 是否选中)	需维码
多选型 (Multi-Select)	IntVar[] (位掩码) 或 BoolVar[]	Map<Integer, BoolVar> (选项ID → 是否选中)	一个辑

如果多选型可选性很多，比如有10000项，这个怎么处理？Mask Color --多选
--10,20,30,40,50
--000000000
--100000000 --10选中
--010000000 --20选中

对于多选型参数有大量选项（如10000项）的情况，直接使用位掩码（bitmask）确实会受到整数位数的限制（32位或64位）。因此，我们需要采用其他方法来高效建模。以下是一种基于OR-Tools SAT的解决方案，使用多个整数变量（IntVar）作为位掩码段（bitmask segments）来扩展表示能力，并结合约束处理。同时，我会提供一个完整的代码示例和测试用例。

请解读Tacton CPQ相关专利和论文，给出器规则系统实现方案，并给出python代码。
“Tacton CPQ专为高度定制化的工业设备（如机械、电气设备等）设计，能管理数千个零件和商业规则，通过少量约束条件生成数百万种有效、可构建的产品变体，同时确保配置符合工程规则与生产可行性”



已思考 (用时 27 秒) ▾

已阅读 9 个网页

Tacton CPQ 的核心在于其规则引擎与约束求解系统，能够高效处理工业级复杂产品的配置问题。下面结合其专利与论文思想（基于公开技术文档分析）、规则系统实现方案，并给出 Python 简化代码实现。

一、Tacton CPQ 规则系统核心技术解析

1. 基于属性的约束系统 (Attribute-Based Constraints)

- 原理：将产品分解为可配置属性（如电机功率、冷却系统），通过逻辑规则（IF-THEN、互斥、依赖）限定属性组合的有效性。例如：“若电机功率 $\geq 100\text{kW}$ ，则冷却系统必须升级”。
- 优势：用少量规则控制指数级变体空间，避免无效配置 1 6。

2. 需求映射技术 (Needs-Based Configuration)

- 原理：将客户业务需求（如“产能 $\geq 50\text{吨/小时}$ ”）自动转换为技术参数，通过问卷引导用户输入，降低销售端技术门槛 1 8。

3. 动态约束传播 (Constraint Propagation)

- 原理：当用户选择一个配置项时，系统实时计算其对其他选项的影响，并立即缩小可选范围（如选择某型号电机后，仅显示兼容的齿轮箱） 5 7。

- 深入探索：问题更加深入，能够触及技术本质

请根据下列数据，使用约束求解器（ORtools），来解决，假设配置的数量不是0，可能是1，或其他大于0的整数，约束很复杂 \n参考的数据： \n "parent_id": "",
"id": "server_001",
"type": "MODULE",
"name": "简单智能服务器",
"category": "server",
"description": "高性能企业级服务器"

以下是用Java实现基于OR-Tools的配置求解器代码，完整解决您提供的复杂约束问题：

java

复制 下载

```
import com.google.ortools.sat.CpModel;  
import com.google.ortools.sat.CpSolver;  
import com.google.ortools.sat.CpSolverStatus;  
import com.google.ortools.sat.IntVar;  
import com.google.ortools.sat.LinearExpr;  
import com.google.ortools.Loader;  
  
public class ServerConfigurationSolver {  
  
    public static void main(String[] args) {  
        // 加载OR-Tools本地库
```

上面的场景我想优化一下 1、cpu和内存、主板都很多 2、当前是以价格最优，我需要提取最优因子，比如货期最优等，请泛化为一个通用方法

复制 编辑

已思考 (用时 114 秒) ✓

通用服务器配置优化框架

下面我将设计一个基于OR-Tools的通用服务器配置优化框架，支持多部选择和多目标优化（成本、货期等）。这个框架可以灵活扩展，支持任意数量的部件类别和优化目标。

java

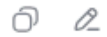
复制 下载

```
import com.google.ortools.sat.*;  
import java.util.*;
```

• 识别AI的局限性：

- AI有时也会不专业，需要从源头分析问题
- 当AI的回答总是不正确时，需要反思问题本身或调整提问方式

在配置器领域，都说基于约束求解器，没有方向性（先配置a，在配置b，在配置c，或先b，在c），那么基于规则引擎是否也是无方向性，请解释



🔍 已深度思考 (用时 11 秒) ▾

• 好的，用户问的是在配置器领域，基于规则引擎是否和基于约束求解器一样没有方向性。首先得确认用户可能是在做技
简单直接的回答是：**不，基于规则引擎的配置在本质上是具有方向性的。**这正是它与基于约束求解器
(Constraint Solver) 的无方向性方法最根本的区别之一。

下面我将详细解释为什么，并对比两者的工作机理。

1. 基于约束求解器 (Constraint Solver)：真正的无方向性

工作原理：

约束求解器将整个配置问题建模为一个**约束满足问题 (CSP, Constraint Satisfaction Problem)**。它包含三个部分：

- **变量 (Variables)**：需要配置的选项（如：发动机型号、颜色、轮胎尺寸）。
- **值域 (Domains)**：每个变量可能的取值范围（如：颜色={红, 白, 蓝}）。
- **约束 (Constraints)**：变量之间必须满足的关系（如：IF 发动机=V8 THEN 变速箱=自动；豪华套装 REQUIRES 真皮座椅）。

无方向性的体现：

当用户做出一个选择时（例如，先选择“颜色=红色”），求解器并不是顺着一个预定的流程去检查后续选项。而是：

1. 将“颜色=红色”这个选择作为一个新的约束，添加到整个约束网络中。
2. 然后，**重新审视整个网络**，根据所有已知约束，**并行地、同时地**推导出所有其他变量的可能状态。



实践要点：

- 提升提问的专业性和深度
- 学会识别AI回答中的错误或不准确之处
- 从源头分析问题，而非盲目接受AI的输出

使用Ortools对一个配置问题进行建模，但是计算结果和预期的不同，怎么定位？请举例说明定位过程，主要使用最新的9.12.4544版本，不要使用老版本

3.3 约束松弛调试

当问题被判定为不可行时，逐步松弛约束以识别冲突：

python

复制 下载

问题描述

java

```
// 变量
x ∈ [0, 10], y ∈ [0, 10]

// 规则
规则1: x + y ≥ 15
规则2: x ≤ 5
规则3: y ≤ 8
```

1. 添加松弛变量

java

```
// 为每个规则添加松弛变量
松弛1 = 布尔变量(规则1可松弛)
松弛2 = 布尔变量(规则2可松弛)
松弛3 = 布尔变量(规则3可松弛)

// 带松弛的约束
x + y ≥ 15 OR 松弛1=true
x ≤ 5 OR 松弛2=true
y ≤ 8 OR 松弛3=true

// 目标: 最小化松弛的规则数量
最小化: 松弛1 + 松弛2 + 松弛3
```

2. 求解结果

text

最优解: x=7, y=8
松弛1=0 (规则1保持)
松弛2=1 (规则2松弛)
松弛3=0 (规则3保持)
目标值=1 (松弛1个规则)

怀疑addConditionalConstraintWithRelaxation方法实现有问题，运行上述程序，得到一个解是 x = 5
y = 0
z = 2，很明显这个解是不满足c3约束的，请修正

上述函数实现还是错的，运行上述代码，得到结果如下，很明显，结果是不满足约束2的，###运行结果: \n
求解状态: OPTIMAL
需要松弛的约束数量: 1

后期阶段：规范AI输出，提升效率，做好综合质量控制

核心策略：TDD驱动，AI作为高级程序员，

项目后期，团队对技术已经较为熟悉，此时AI的角色转变为高效的操作员：

- **规范AI的想法**：通过TDD（测试驱动开发）确保代码质量
 - **让AI做重复性工作**：如文档修改、代码重构等
- TDD驱动，自动化测试框架作为关键需求，确保AI生成的代码不影响核心功能

通过业务注解，自动生成基础数据，5行注解->160行json数据，方便代码阅读



流水业务比较器，仅关注业务结果，3行代码1个用例，方便阅读

```
/**
 * 测试只有一个解的情况
 */
@Test
public void testOnlyOneSolution() {
    // 测试颜色参数推理
    inferParas(partCode: "tShirt11", qty: 1);

    // 验证第一个解
    solutions(index: 0).assertPara(code: "color").valueEqual(expectValue: "Red")
        .assertPara(code: "size").valueEqual(expectValue: "Small");
    printSolutions();
}
```

不仅保证了快速验证了AI生成的代码，同时，也方便新人掌握这块代码。

使用AI做新特性开发(人：提供明确规格)

Prompt:

```
__**User (2025-09-06 09:32Z)**__

请按下列要求增强代码，实现方案如下：
1、在ConstraintAlgImpl增加方法
void autoAddConstraints(Module module) {
    对module.rules的每个rule
        根据rule.code(如:rule1)在当前类找到方法ruleMethod (如: protected void rule2() {})
        可调用ruleMethod，完成添加和注册
}
2、在ConstraintAlgImpl.initConstraint调用autoAddConstraints
```

实现代码：

```
src/main/java/com/jmix/configengine/artifact/ConstraintAlgImpl.java 56-1
170 +  */
171 + protected void autoAddConstraints(Module module) {
172 +     if (module == null || module.getRules() == null || module.getRules().isEmpty()) {
173 +         log.debug("No rules found in module, skipping auto constraint registration");
174 +         return;
175 +     }
176 +
177 +     try {
178 +         // 获取当前类的所有方法
179 +         Method[] methods = this.getClass().getDeclaredMethods();
180 +         Map<String, Method> methodMap = new HashMap<>();
181 +
182 +         // 构建方法名到方法的映射，方便查找
183 +         for (Method method : methods) {
184 +             methodMap.put(method.getName(), method);
185 +         }
186 +
187 +         // 遍历所有规则
188 +         for (Rule rule : module.getRules()) {
189 +             if (rule == null || rule.getCode() == null || rule.getCode().trim().isEmpty()) {
190 +                 continue;
191 +             }
192 +
193 +             String ruleCode = rule.getCode();
194 +             // 构造方法名: addConstraint_ + ruleCode
195 +             String methodName = "" + ruleCode;
196 +
197 +             // 查找对应的方法
198 +             Method ruleMethod = methodMap.get(methodName);
199 +             if (ruleMethod != null) {
200 +                 try {
201 +                     // 设置方法可访问 (处理protected方法)
202 +                     ruleMethod.setAccessible(true);
203 +                     // 调用方法
204 +                     ruleMethod.invoke(this);
205 +                     log.info("Successfully registered constraint for rule: {}", ruleCode);
206 +                 } catch (Exception e) {
207 +                     log.error("Failed to invoke rule method: {} for rule: {}", methodName, ruleCode, e);
208 +                 }
209 +             }
210 +         }
211 +     }
212 + }
```

使用AI快速做重构—复杂逻辑抽取（人：提供重构线索）

重构前：

Prompt：“把选择的代码参考addCompatibleConstraintRequires泛化抽取为一个父类的函数Incompatible”

```
51 + protected void addCompatibleConstraintIncompatible_note() {
52 +     // 创建条件变量: A在CA中
53 +     BoolVar inCA = model.newBoolVar("inCA");
54 +     model.addBoolOr(new Literal[]{
55 +         AVar.getParaOptionByCode("a1").getIsSelectedVar(),
56 +         AVar.getParaOptionByCode("a3").getIsSelectedVar()
57 +     }).onlyEnforceIf(inCA);
58 +     model.addBoolAnd(new Literal[]{
59 +         AVar.getParaOptionByCode("a1").getIsSelectedVar().not(),
60 +         AVar.getParaOptionByCode("a3").getIsSelectedVar().not()
61 +     }).onlyEnforceIf(inCA.not());
62 +
63 +     // 创建条件变量: B在CB中
64 +     BoolVar inCB = model.newBoolVar("inCB");
65 +     model.addBoolOr(new Literal[]{
66 +         BVar.getParaOptionByCode("b1").getIsSelectedVar(),
67 +         BVar.getParaOptionByCode("b2").getIsSelectedVar(),
68 +         BVar.getParaOptionByCode("b3").getIsSelectedVar()
69 +     }).onlyEnforceIf(inCB);
70 +     model.addBoolAnd(new Literal[]{
71 +         BVar.getParaOptionByCode("b1").getIsSelectedVar().not(),
72 +         BVar.getParaOptionByCode("b2").getIsSelectedVar().not(),
73 +         BVar.getParaOptionByCode("b3").getIsSelectedVar().not()
74 +     }).onlyEnforceIf(inCB.not());
75 + }
```

父类新增函数：

```
src/main/java/com/jmix/configengine/artifact/ConstraintAlgImpl.java 74
387 + * 兼容性规则: Incompatible关系约束
388 + * 规则内容: 双向不兼容关系, 左侧参数和右侧参数不能在对应的组内同时存在
389 + * 例如: (a1,a3) Incompatible (b1,b2,b3) 表示:
390 + * - 如果A选择a1或a3, 则B不能选择b1、b2或b3 (必须选择b4或b5)
391 + * - 如果A选择a2、a4或a5, 则B可以选择任意值
392 + * - 如果B选择b1、b2或b3, 则A不能选择a1或a3 (必须选择a2、a4或a5)
393 + * - 如果B选择b4或b5, 则A可以选择任意值
394 + */
395 + public void addCompatibleConstraintIncompatible(String ruleCode, ParaVar leftParaVar, List<String> leftParaFilterOptionCodes,
396 + ParaVar rightParaVar, List<String> rightParaFilterOptionCodes) {
397 + // left: 确保只有一个参数选项被选中
398 + model.addExactlyOne(leftParaVar.optionSelectVars.values().stream()
399 + .map(option -> option.getSelectedVar())
400 + .toArray(BoolVar[]::new));
401 + // right: 确保只有一个参数选项被选中
402 + model.addExactlyOne(rightParaVar.optionSelectVars.values().stream()
403 + .map(option -> option.getSelectedVar())
404 + .toArray(BoolVar[]::new));
405 +
406 + // 由于右约束性, 左约束值由至少一个被选中
```

原有类做相应修改并补充测试用例：

```
src/test/java/com/jmix/configengine/scenario/ruletest/CompatibleRuleIncompatibleTest.java 339
29 + @ParaAnno(
30 + options = {"b1", "b2", "b3", "b4", "b5"}
31 + )
32 + private ParaVar BVar;
33 +
34 + @Override
35 + protected void initConstraint() {
36 + //注意: 一定要选择一个值怎么理解 (如果不选择, 也设置一个可选值就可以表达, 这样模型统一了)
37 + //A=(a1,a2,a3,a4,a5), AVar.value的值域
38 + //B=(b1,b2,b3,b4,b5), BVar.value的值域
39 + // 规则内容: (a1,a3) Incompatible (b1,b2,b3), 则CA=(a1,a3), CB=(b1,b2,b3), NCA=(a2,a4,a5), NCB=(b4,b5)
40 + // 解读:
41 + // 1、正向
42 + // 1.1 如果AVar.value取CA的一个值(在CA中), 则BVar.value一定不是CB的一个值, 取CB内的值是非法的, 只能是NCB内的值, 如: 如果AVar.var=a1, 则BVar.var=b4或b5(NCB), 不可以是b1 或 b2 或 b3(CB)
43 + // 1.2 如果AVar.value取NCA一个值(不在CA中), 则BVar.value可以是CB的一个值, 也可以是NCB的一个值, 如: 如果AVar.var=a2, 则BVar.var=b1或b2或b3(CB), 也可以是b4或b5(NCB)
44 + // 2、反向
45 + // 2.1 如果BVar.value取CB的一个值(在CB中), 则AVar.value一定不是CA的一个值, 取CA内的值是非法的, 只能是NCA内的值, 如: 如果BVar.var=b1, 则AVar.var=a2或a4或a5(NCA), 不可以是a1 或 a3(CA)
46 + // 2.2 如果BVar.value取NCB一个值(不在CB中), 则AVar.value可以是CA的一个值, 也可以是NCA的一个值, 如: 如果BVar.var=b4, 则AVar.var=a1 或 a3(CA), 也可以是a2 或 a4或a5(NCA)
47 + // 使用泛化的Incompatible约束方法
48 + addCompatibleConstraintIncompatible("rule1", AVar, Arrays.asList("a1", "a3"), BVar, Arrays.asList("b1", "b2", "b3"));
49 +
50 +
```

```
src/test/java/com/jmix/configengine/scenario/ruletest/CompatibleRuleIncompatibleTest.java
159 +
160 + @Test
161 + public void testInCAWithInCBIsInvalid() {
162 + // 测试正向规则1.1: 如果A在CA中(a1), 则B在CB中(b1、b2或b3)是不合法的
163 + inferParasByPara("A", "a1");
164 + assertSolutionNum("B:b1", 0);
165 + assertSolutionNum("B:b2", 0);
166 + assertSolutionNum("B:b3", 0);
167 + }
168 +
169 + @Test
170 + public void testNotInCAAllowsAnyB() {
171 + // 测试正向规则1.2: 如果A不在CA中(a2), 则B可以是CB或NCB中的任意值
172 + inferParasByPara("A", "a2");
173 +
```

讲数据求解的模型代码的

明确AI的定位：AI是工具，不是决策者

实践要点：

- 使用TDD确保AI生成代码的正确性

- 将重复性、机械性工作交给AI处理
- 保持人的主导地位，AI作为执行工具

四、关键心得

1. 人机协作的定位

- **人**：负责决策、架构设计、质量把控、问题分析
- **AI**：负责代码生成、文档编写、重复性工作、知识查询

2. 不同阶段的AI角色

- **初期**：导师（需要验证）
- **中期**：伙伴（需要识别错误）
- **后期**：操作员（需要规范输出）

3. 成功的关键因素

- **验证一切**：对AI的输出必须验证，不能盲目信任
- **持续学习**：通过关联学习法快速建立知识框架
- **专业提问**：提问质量决定回答质量
- **TDD保障**：测试驱动开发确保代码质量

五、总结

在新技术项目中，AI可以显著提升开发效率，但必须明确人机协作的定位。人是决策者和质量把控者，AI是执行工具和知识助手。通过在不同阶段采用不同的协作策略，我们成功在1.5个月内完成了原本需要3-6个月的项目，证明了AI辅助开发在技术穿刺项目中的巨大价值。

核心启示：AI不是替代者，而是放大器。它放大了我们的学习能力、编码能力和问题解决能力，但前提是我们必须保持批判性思维，持续验证，并明确自己的主导地位。